

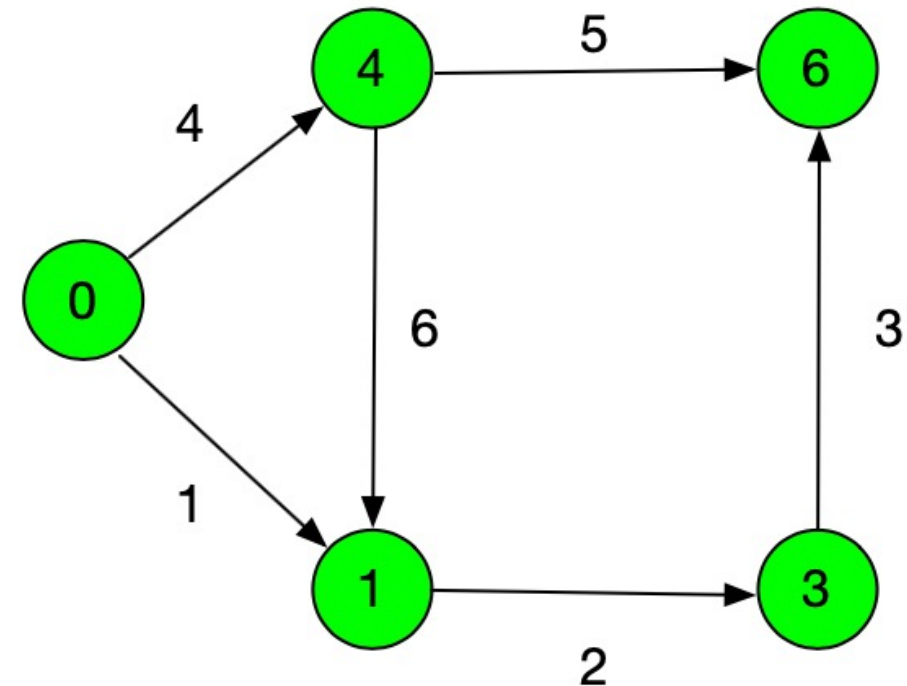
An Adaptive Asynchronous Approach for the Single- Source Shortest Paths Problem

Ritvik Rao, Kavitha Chandrasekar, Laxmikant Kale



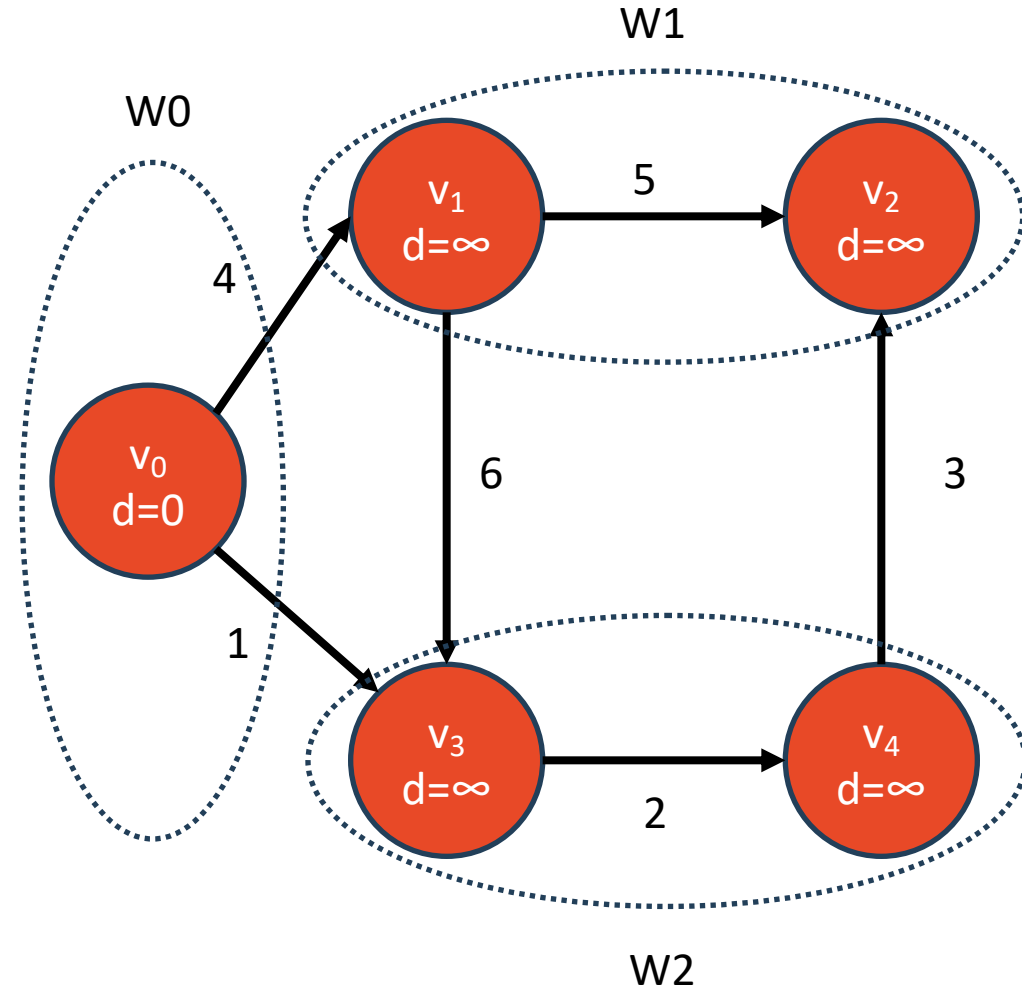
Single-Source Shortest Paths (SSSP)

- Large-scale graphs must be processed in parallel
- Label-setting (e.g. Dijkstra)
 - Forces distance order, decreasing parallelism
- Label-correcting
 - Set multiple distances at once, correct later
- Focus on label-correcting



Parallel Bellman-Ford

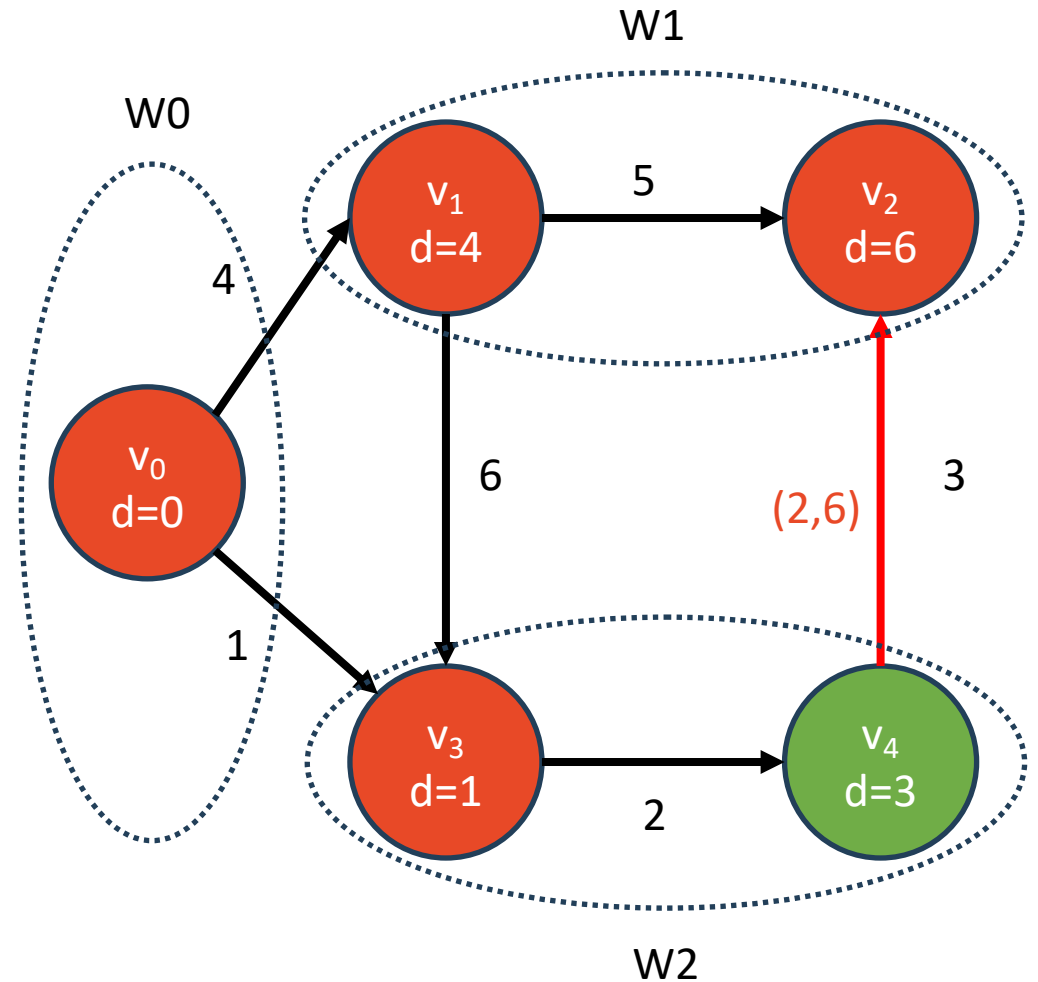
- Update = (v, c)
 - v = target vertex
 - c = cost of some path to v



Parallel Bellman-Ford

- Total number of updates: 6
 - v_2 updated twice
- Fully asynchronous
- High parallelism

- **Problem: Wasted Work**
- We only care about the final distance value

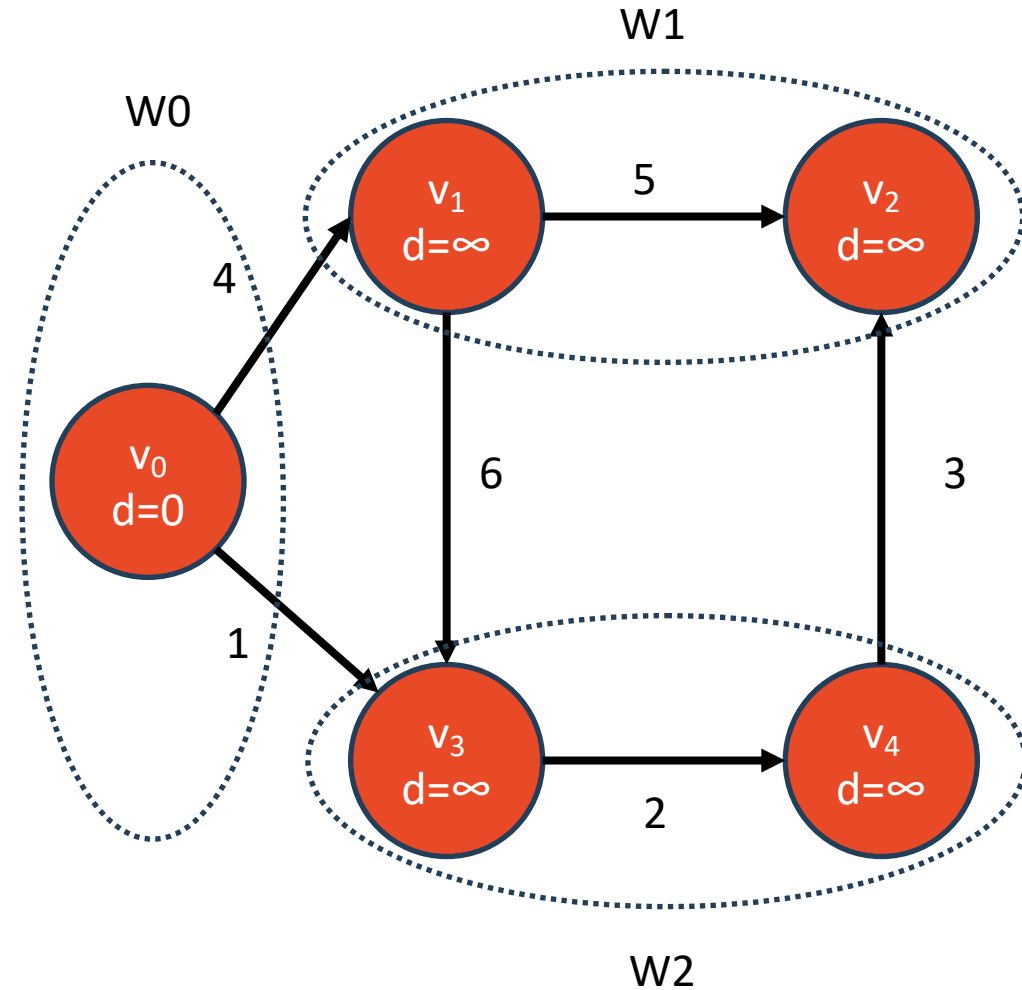


Delta-stepping

- Parallelism, only within a certain range

$\Delta=3$

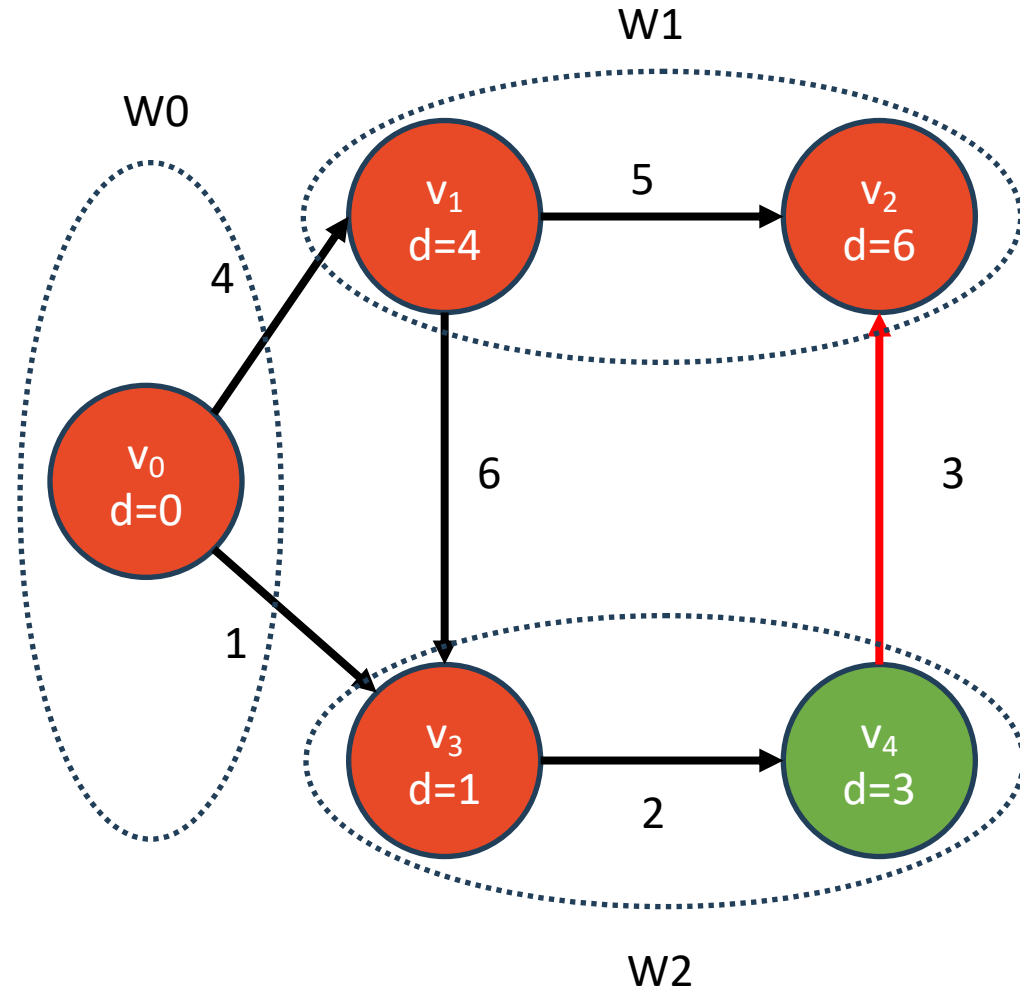
Bucket number (range)	0 (0-2)	1 (3-5)	2 (6-8)
Vertices in bucket	v_0		



Delta-stepping

$\Delta=3$

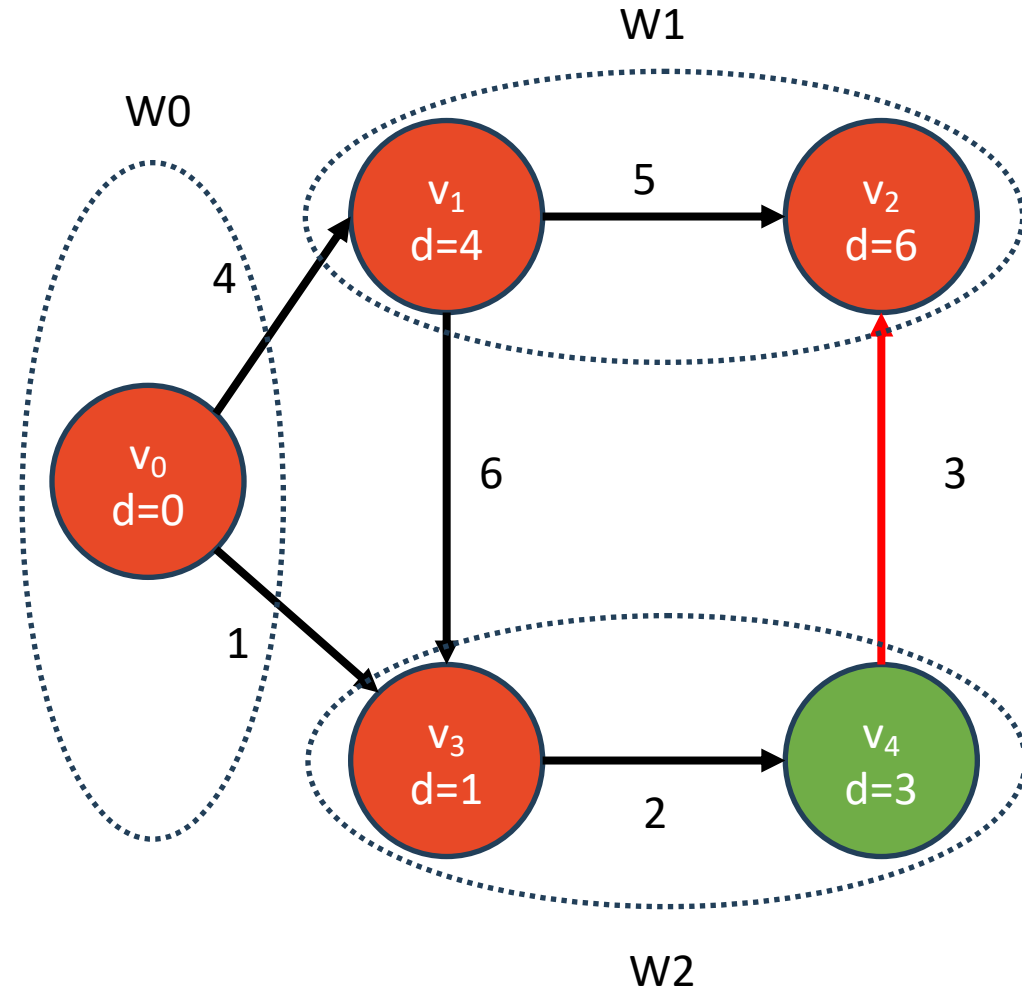
Bucket number (range)	0 (0-2)	1 (3-5)	2 (6-8)
Vertices in bucket	v_0, v_3	v_4, v_1	v_2



Delta-stepping

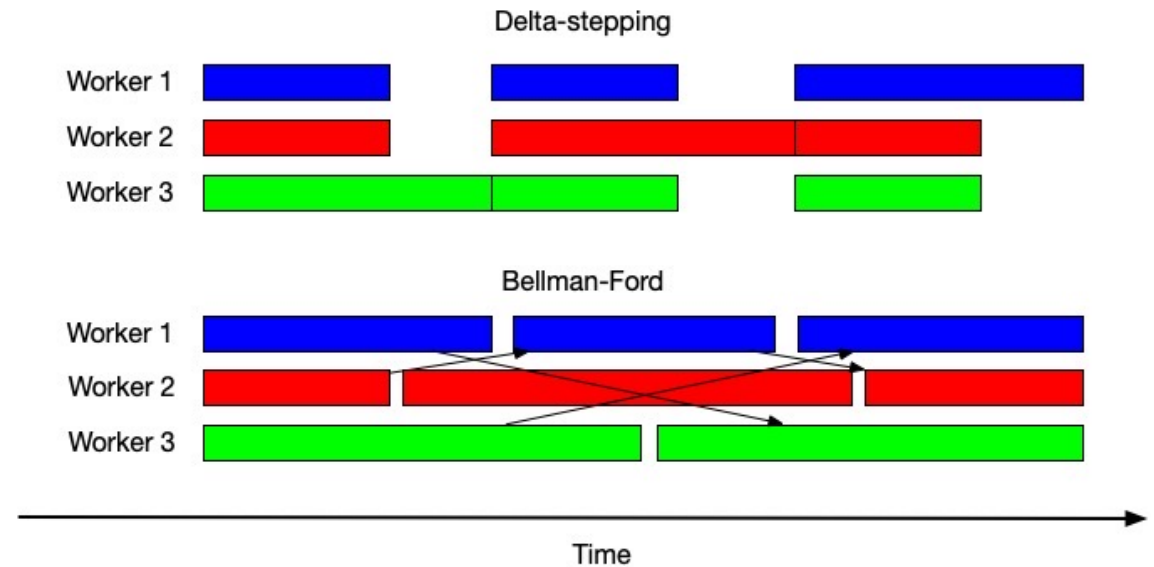
- Total number of updates: 5
- Bulk synchronous

- **Problem: Synchrony**
- All workers must complete bucket and wait for others



Synchronization vs. work

- Delta-stepping:
Synchronization
- Bellman-Ford: Wasted
work



Synchronization vs. work

Can we do better?

Yes: Control communication during algorithm



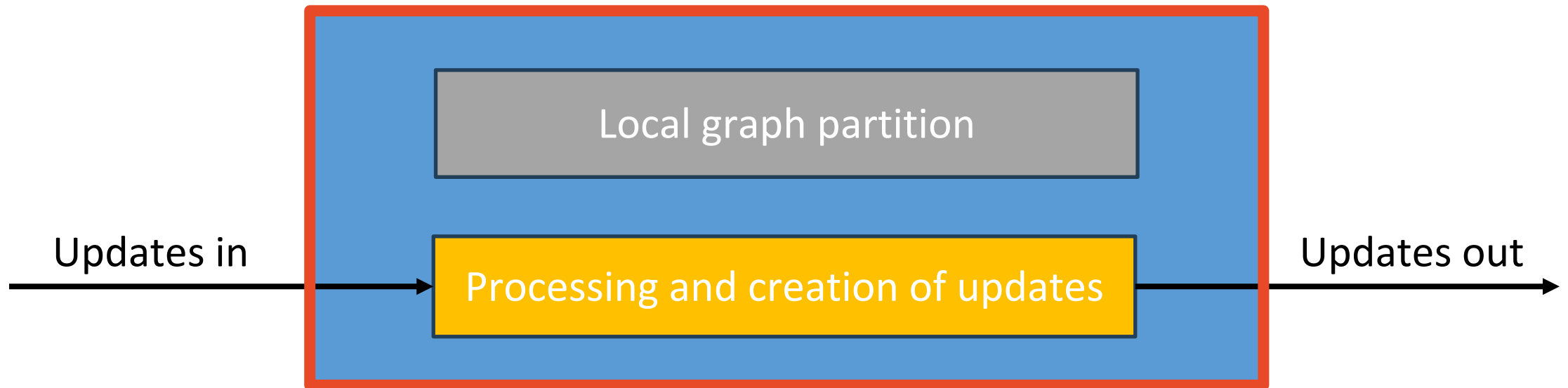
ACIC

- **A**synchronous **C**ontinuous **I**ntrospection and **C**ontrol
- New runtime technique for fine-grained applications



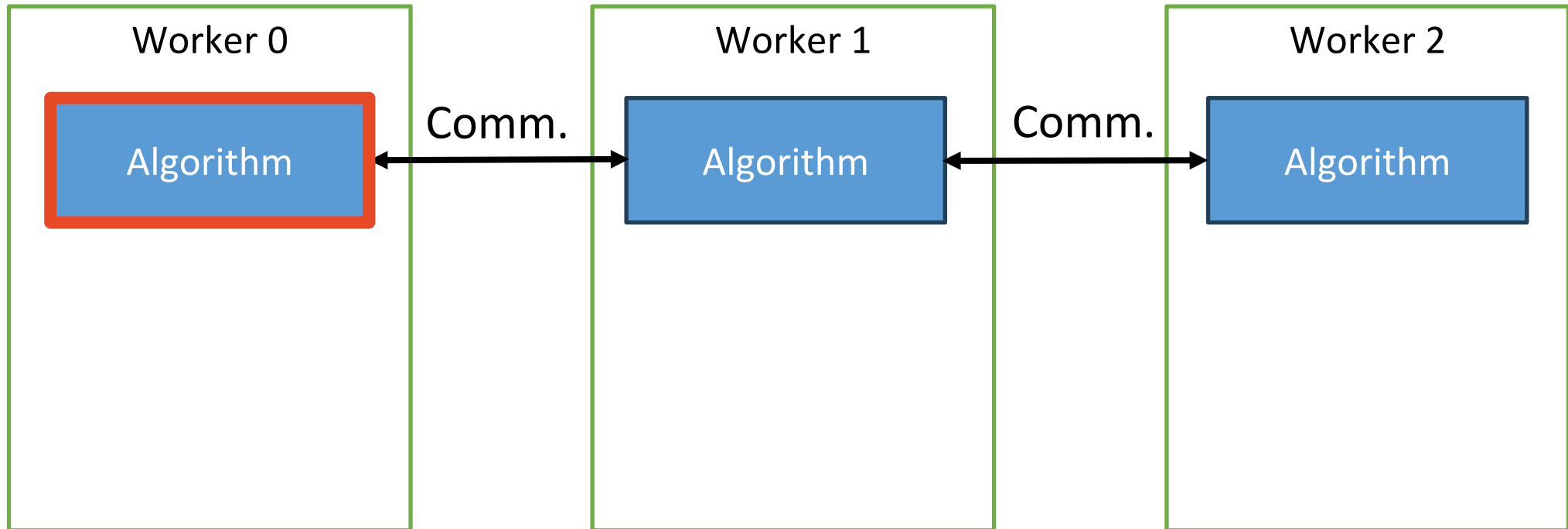
ACIC

- **Asynchronous:** Asynchronous, fine-grained, communication-driven applications
 - In this example: Parallel Bellman-Ford



ACIC

- **Asynchronous:** Asynchronous, fine-grained, communication-driven applications
 - Workers execute and communicate independently



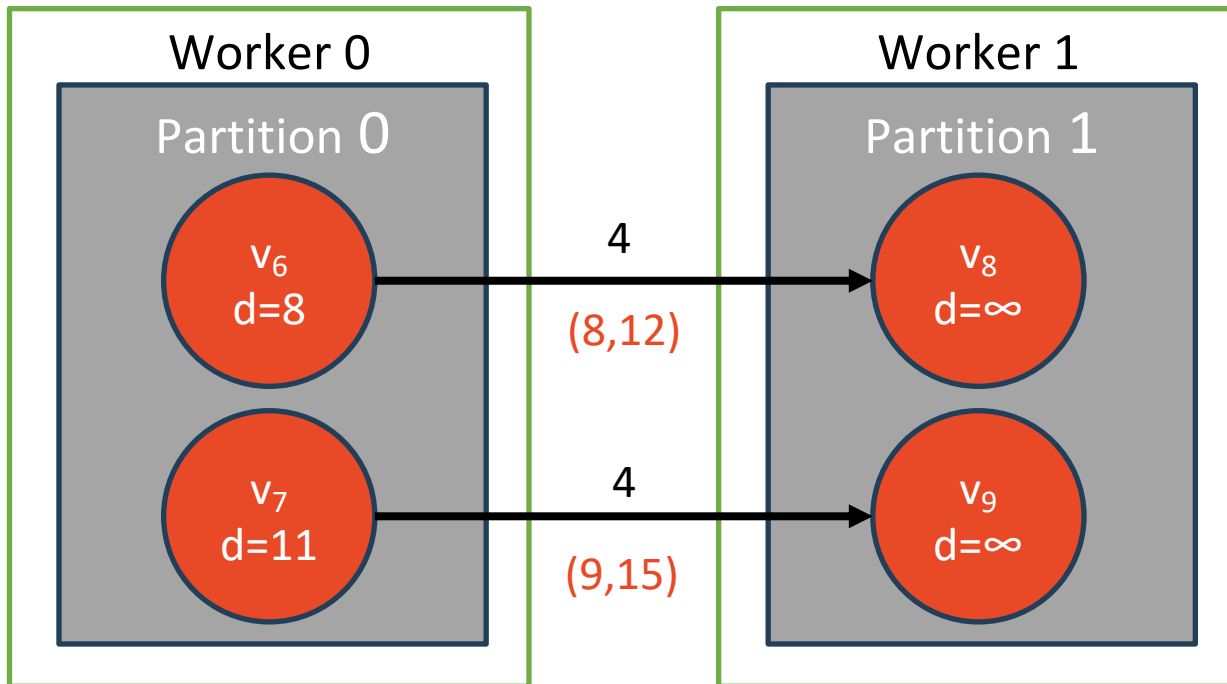
ACIC

- **Introspection: Reduction on control parameters**
 - Control parameters: State/snapshot of progress at a given point
 - For SSSP: an **update histogram** for the distribution of active updates
 - Why?
 - The **lower** the update cost, the more likely it's the **final answer**
 - **Every** reduction in a cost value leads to new updates
 - **The fewer times a vertex distance is updated, the fewer updates are created**
 - **Less work, faster termination**



ACIC

- Introspection: Reduction on control parameters
 - For SSSP: an **update histogram** for the distribution of active updates

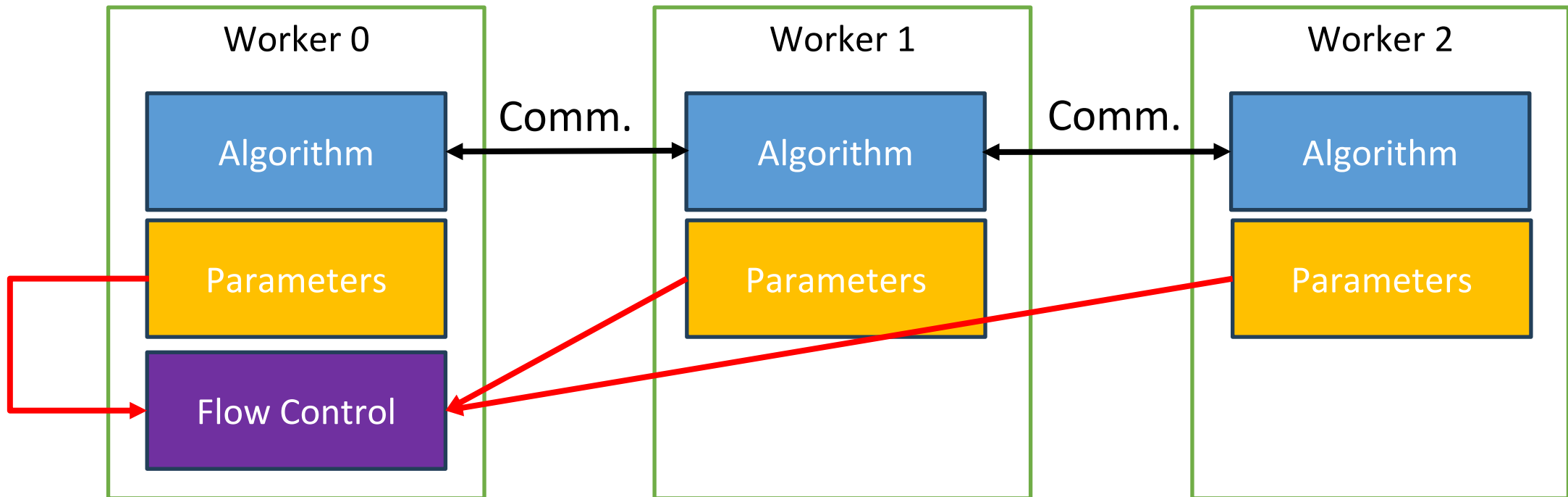


Distance range	0-2	3-5	6-8	9-11	12-14	15-17	18-20	21-23
# updates	0	1	2	4	1	1	2	0

Distance range	0-2	3-5	6-8	9-11	12-14	15-17	18-20	21-23
# updates	0	0	1	2	5	5	3	0

ACIC

- **Introspection: Reduction on control parameters**
 - Sum up all parameters, place result on worker 0



ACIC

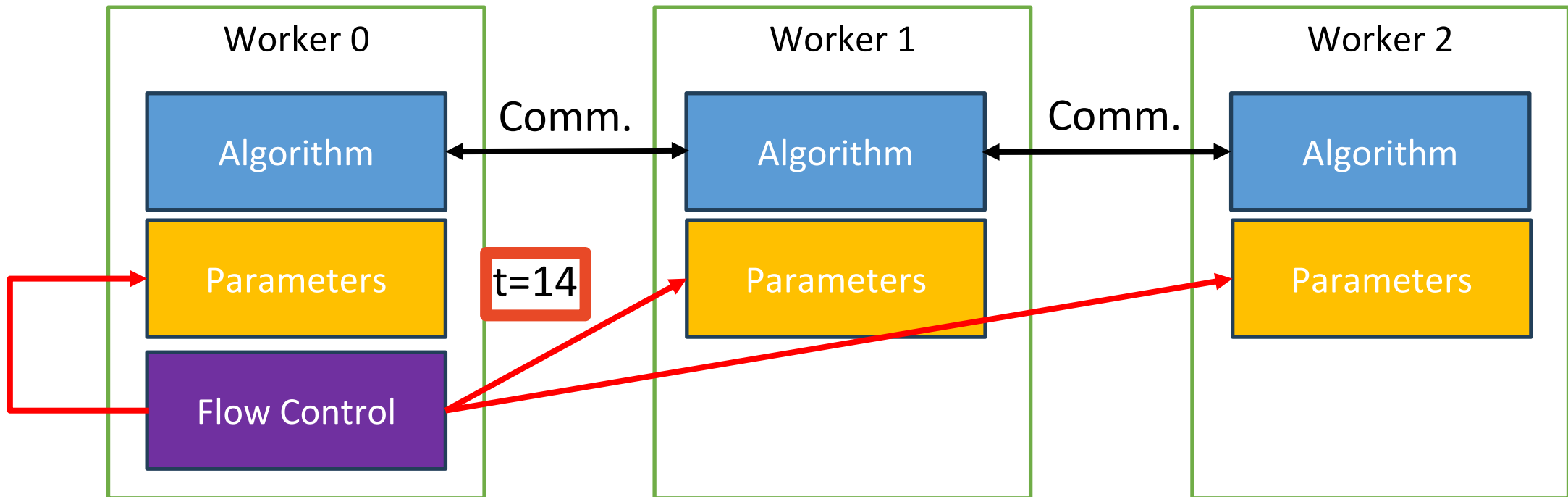
- **Control:** Prioritize communication using a threshold
 - Based on combined control parameters
 - E.g. threshold = 50% of updates
 - Therefore, set threshold=14

Combined histogram								
Distance range	0-2	3-5	6-8	9-11	12-14	15-17	18-20	21-23
# updates	0	1	3	6	6	6	5	0
Running total	0	1	4	10	16	22	27	27



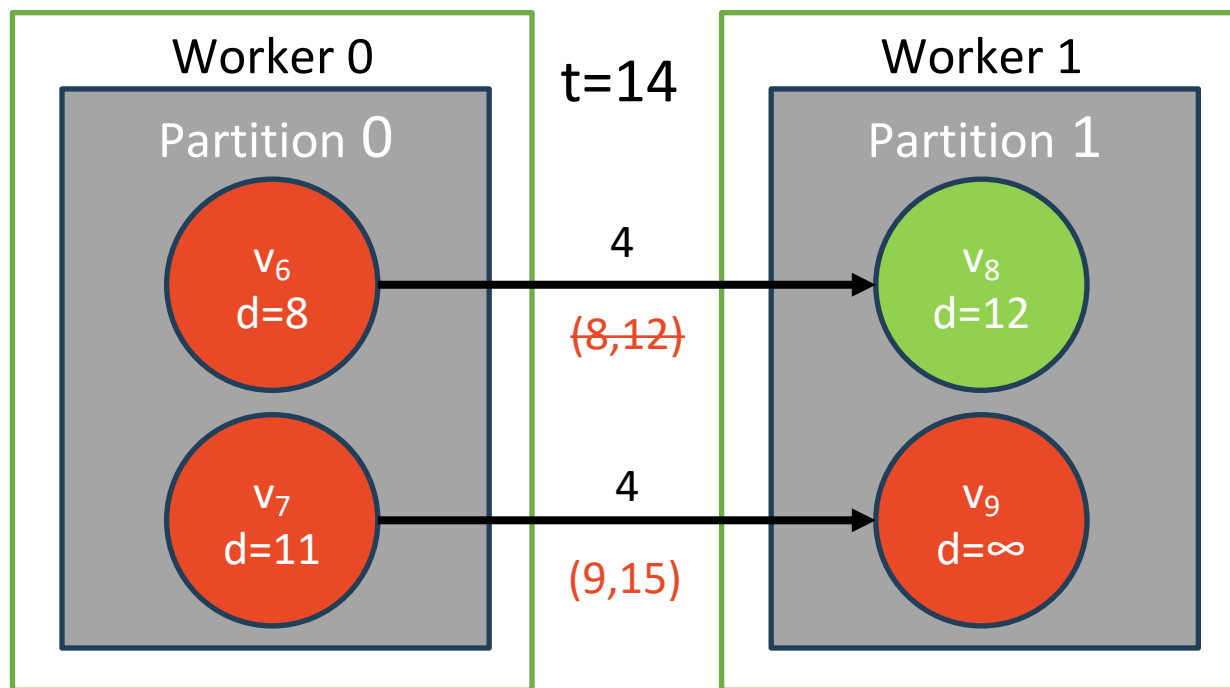
ACIC

- **Control:** Prioritize communication using a threshold
 - Broadcast threshold to workers



ACIC

- **Control:** Prioritize communication using a threshold
 - Update (8,12) is used, so decrement histogram on worker 1



Worker 0 histogram

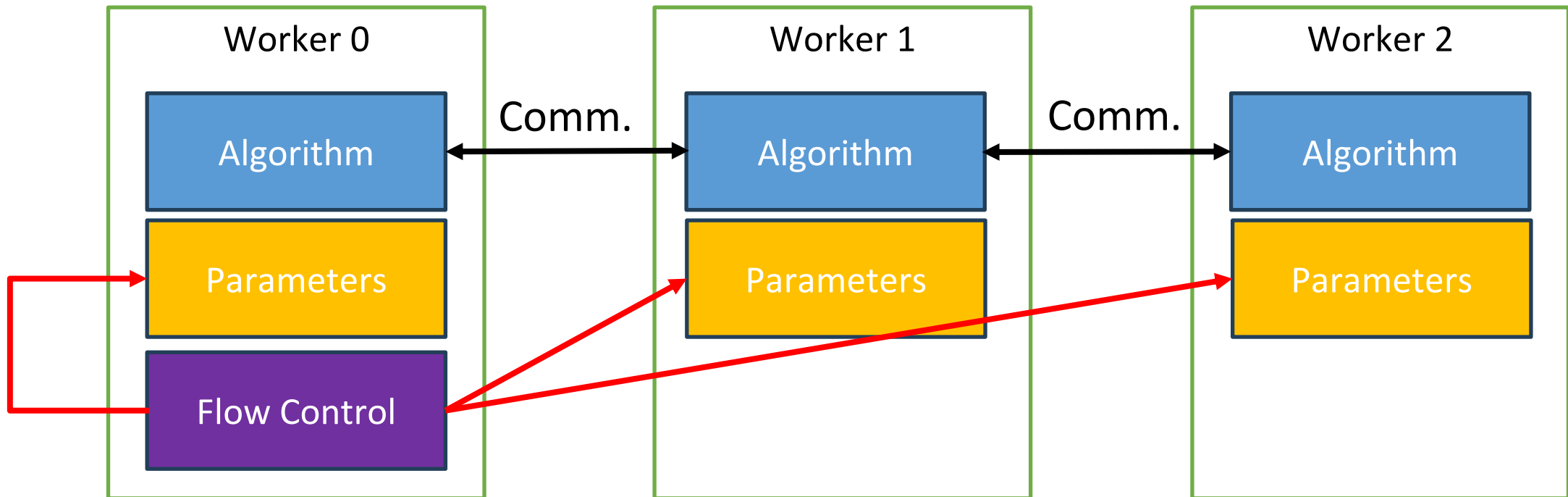
Distance range	0-2	3-5	6-8	9-11	12-14	15-17	18-20	21-23
# updates	0	1	2	4	1	1	2	0

Worker 1 histogram

Distance range	0-2	3-5	6-8	9-11	12-14	15-17	18-20	21-23
# updates	0	0	1	2	4	5	3	0

ACIC

- **Continuous:** Repeatedly reduce/broadcast until termination
 - Like "heartbeat" as algorithm executes



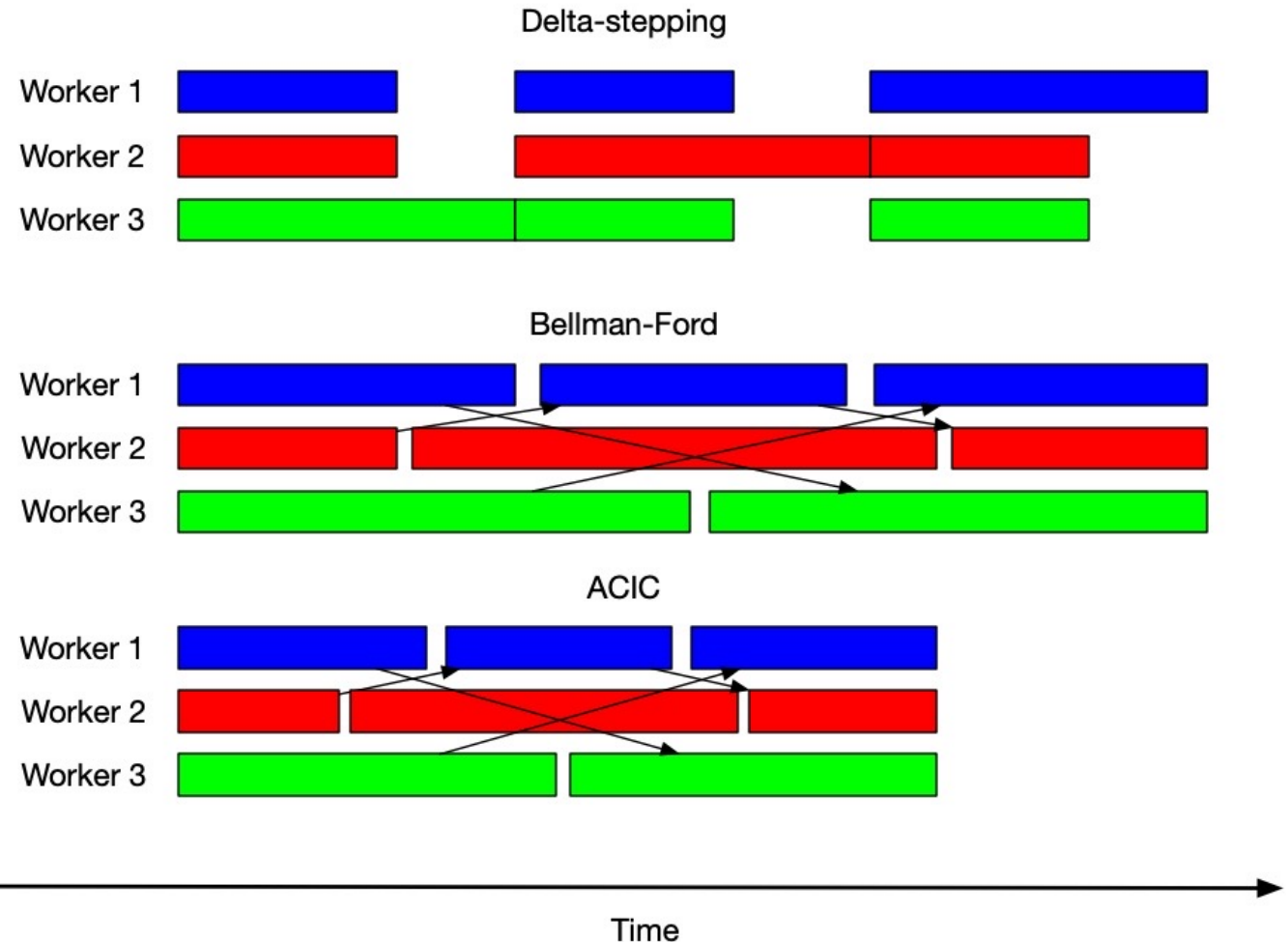
ACIC: Other optimizations

- Priority queue and threshold for generation of updates
 - Process incoming (aggregated) updates as soon as possible, then use priority queue up to a given threshold
- Message aggregation to reduce overhead for fine-grained tasks
 - Use Tramlib to control size of buffers and patterns for aggregation



ACIC

- Control parameters allow for adaptivity
 - Key to reducing work
- Concurrent with algorithm
 - Asynchronous
- Low overhead broadcasts and reductions



ACIC

- **In short:**
 - Asynchronous applications
 - Continuous operation during execution
 - Introspection with reduction on control parameters
 - Control with broadcasted threshold, because **update processing order defines total work created**

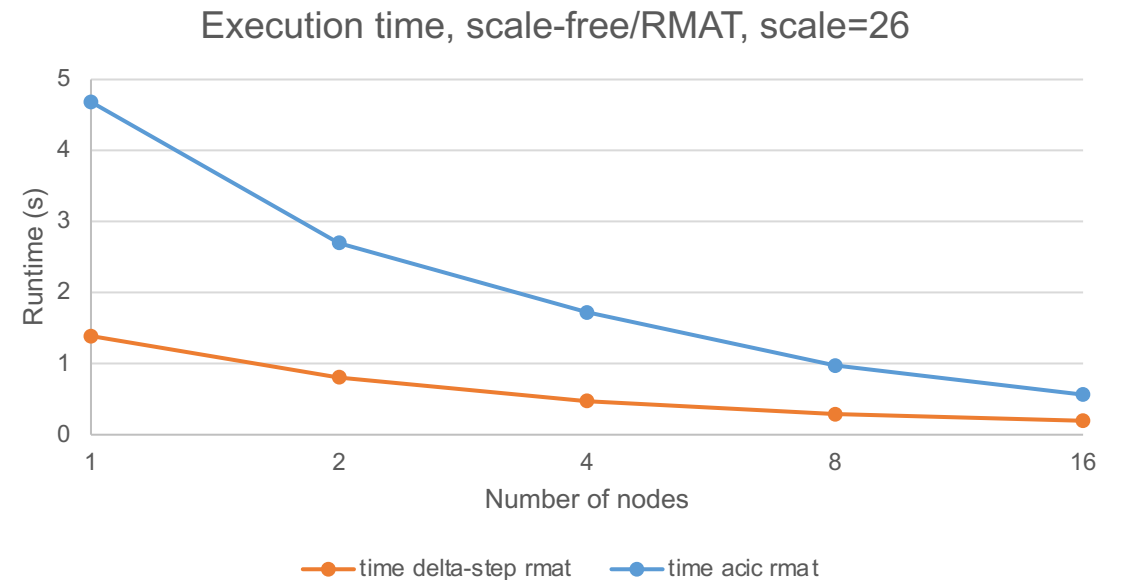
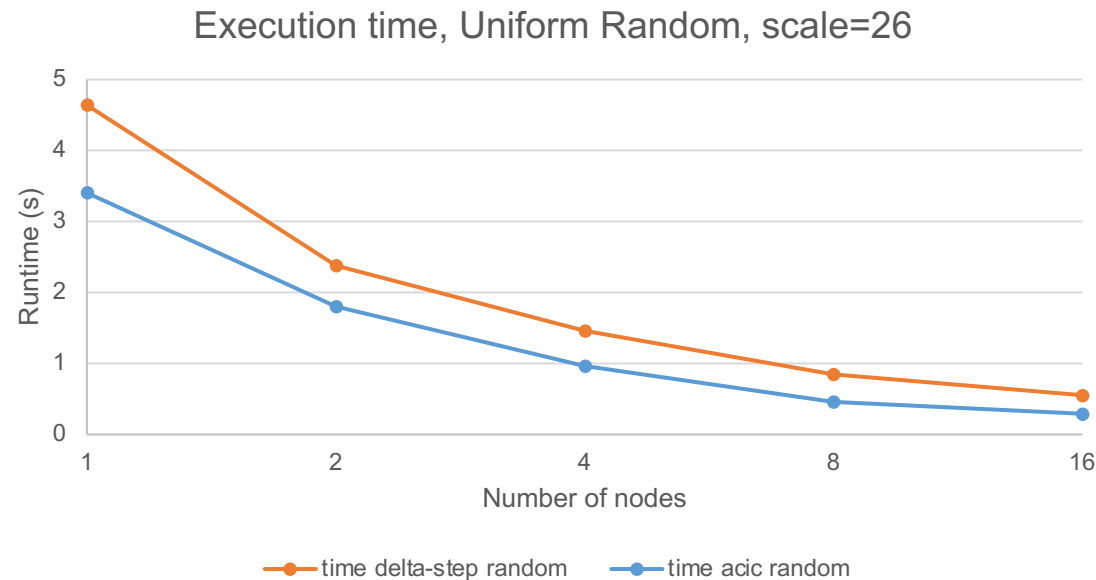


Experimental Setup

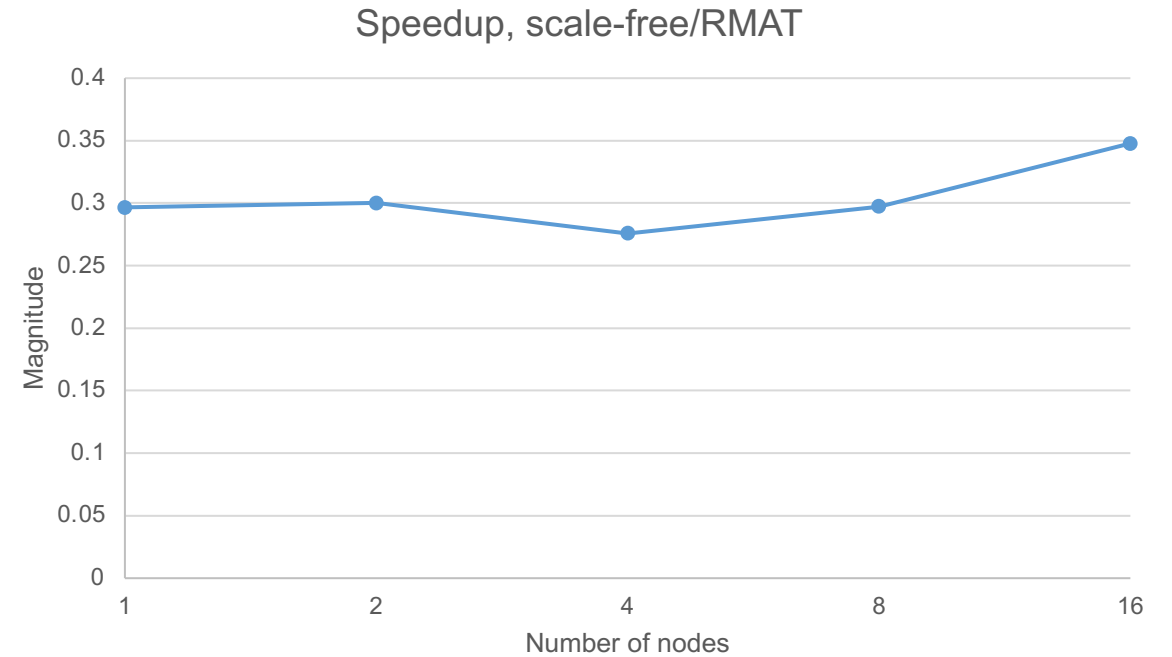
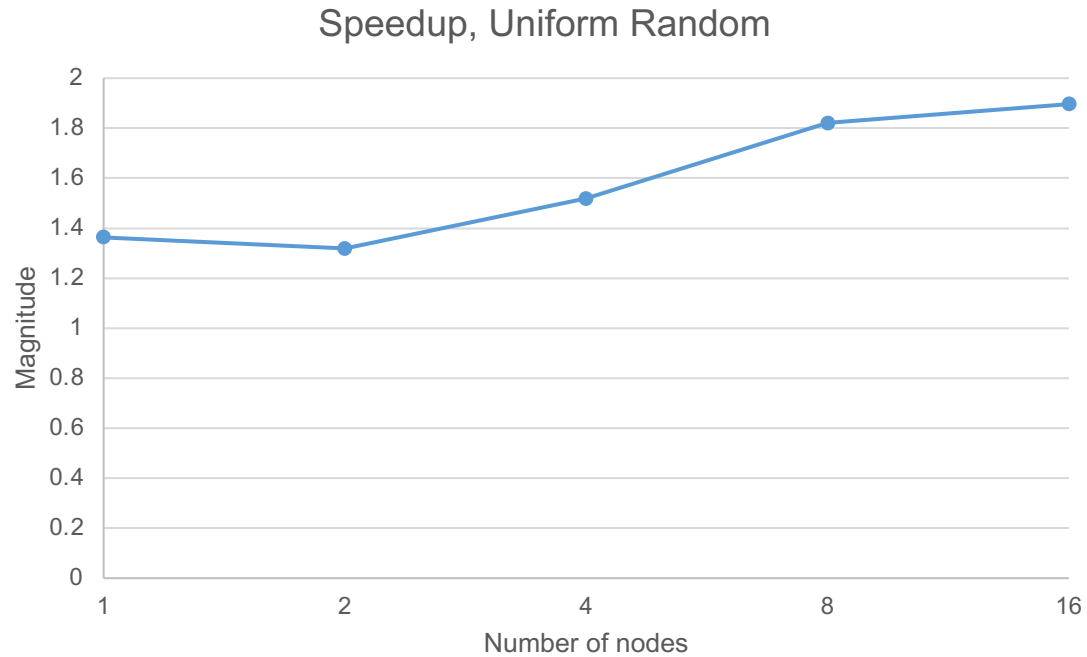
- Scale-free graph ($|V|$ scale=26, degree=16)
 - generated with RMAT method
- Uniform random graph ($|V|$ scale=26, degree=16)
 - For any given edge, select origin and destination uniformly and randomly
- Frontier supercomputer
 - 48 workers per node, 6 workers per process
- Modified Delta-stepping implementation
 - 2D partitioning
 - Direction optimization



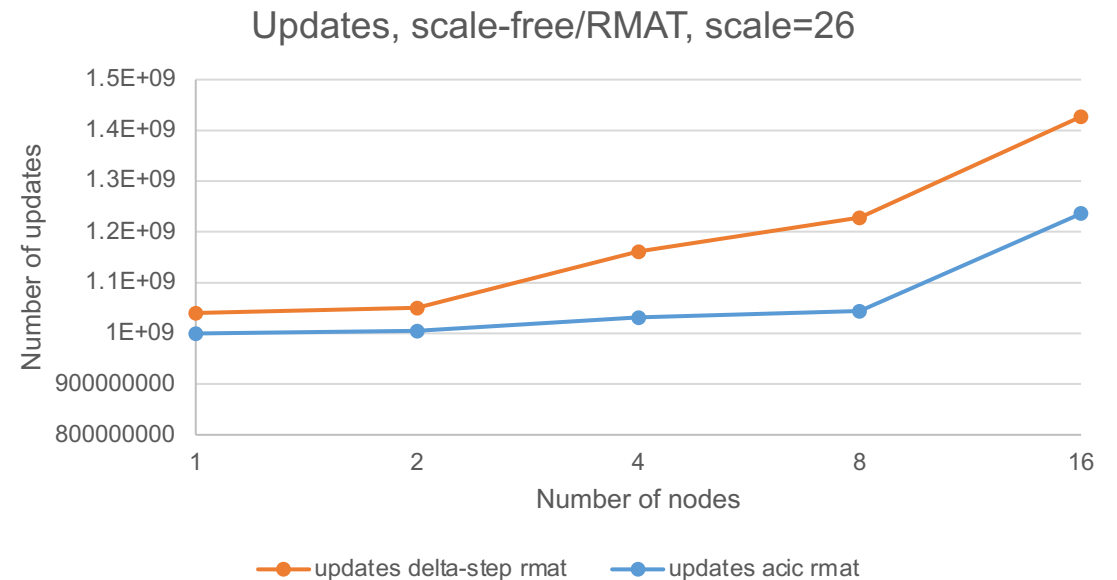
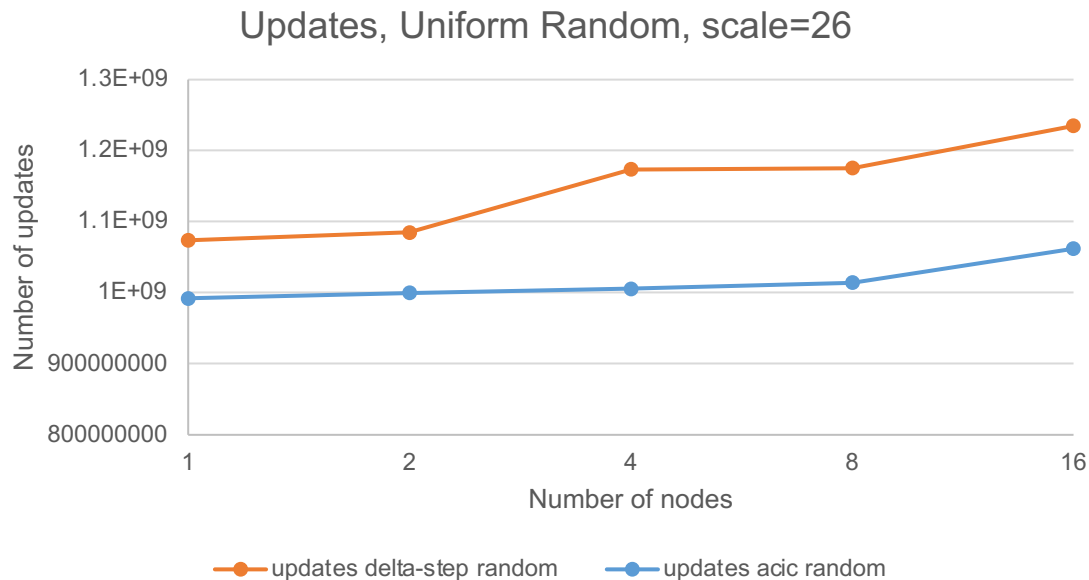
Results: Execution time



Results: Speedup



Results: Updates/Work



Conclusion

- Adaptive introspection model for fine-grained asynchronous algorithms
- Application to SSSP
- Performs well on uniform random graphs
- Future work:
 - Run on high-diameter graphs
 - Implement load-balancing strategies
 - Try other graph algorithms with high synchronization costs



Questions?

- More details available in our paper

An Adaptive Asynchronous Approach for the Single-Source Shortest Paths Problem

Ritvik Rao
University of Illinois
Urbana, IL, USA
rsrao2@illinois.edu

Kavitha Chandrasekar
University of Illinois
Urbana, IL, USA
kchndrs2@illinois.edu

Laxmikant Kale
University of Illinois
Urbana, IL, USA
kale@illinois.edu

Abstract—Large-scale graphs with billions and trillions of vertices and edges require efficient parallel algorithms for common graph problems, one of which is single-source shortest paths (SSSP). Bulk-synchronous parallel algorithms such as Δ -stepping experience large synchronization costs at the scale of many nodes, so asynchronous approaches are needed for scalability. However,

method. However, this approach leads to many unnecessary updates because the application has no global view of the distance value distribution of updates, and therefore no control over the flow of messages based on distance value.

A compromise between Δ -stepping and distributed control

